

INFSCI 1540 Final Report

Retail Sales Data Warehouse Using Superstore Dataset

A Technical Report submitted to Vladimir Zadorozhny
at the School of Computing and Information
University of Pittsburgh — Pittsburgh, Pennsylvania

Spring 2025-2026

Project Team Members

Nate Moser
Srivats Pulumati
Justin Bailey

Abstract

This project presents the design and implementation of a retail sales data warehouse built on top of the Superstore dataset. The system uses Kafka for real-time data streaming, MySQL for both operational and warehouse storage, and a star schema to organize data for analysis. Once the data was loaded and transformed into fact and dimension tables, we ran OLAP queries to pull out business insights — things like which regions drive the most sales, which product categories are most profitable, and how sales trend over time month to month.

Introduction

Data warehouses are a core part of how modern businesses make sense of their data. Instead of querying raw transactional records every time you need an answer, a well-designed warehouse organizes everything into a structure built for analysis — fast, consistent, and easy to query at scale.

For this project, we built a full retail data warehouse using a streaming pipeline. The goal was to simulate how data would actually flow in a real system: transactions come in, get stored in an operational database, and then get transformed into a structured warehouse that supports analytical queries. We used Kafka to handle the streaming side, MySQL to store everything, and a star schema to design the warehouse.

The dataset we used is the Superstore retail dataset from Kaggle, which contains around 9,994 real-looking sales transactions across different regions, product categories, and customer segments. It was a good fit for this kind of project because it has enough variety to make the OLAP queries genuinely interesting.

1. Data Description

The Superstore dataset is a retail dataset from Kaggle that simulates sales for a fictional company operating across the United States. It has 9,994 records, each representing a single order line item. The fields we used were:

- Order ID — unique identifier for each order
- Order Date — when the order was placed
- Customer Name — name of the customer who placed the order
- City, State, Region — where the order was shipped
- Product Name, Category — what was ordered
- Sales — revenue from the transaction
- Quantity — number of units ordered

- Profit — profit generated from the transaction

We left out fields that weren't relevant to our warehouse design — Ship Date, Ship Mode, Segment, Postal Code, Sub-Category, and Discount. Keeping the schema focused made the pipeline simpler and the queries cleaner.

2. Data Preparation

The dataset was pretty clean out of the box — no missing values, no obvious duplicates, and the structure was consistent. The one issue we ran into was the date format.

The original CSV stores Order Date as MM/DD/YYYY (e.g. 11/8/2016), but MySQL's DATE type expects YYYY-MM-DD. Loading the raw format would either cause an error or silently store null. We fixed this in the Kafka consumer with a one-line conversion before insertion:

```
raw_date = data.get('Order Date')
formatted_date = datetime.strptime(raw_date,
    '%m/%d/%Y').strftime('%Y-%m-%d')
```

That was the only preprocessing step needed. Everything else loaded cleanly.

3. Docker Containers

We used Docker to run all the infrastructure so we didn't have to install Kafka or MySQL directly on our machines. The docker-compose.yml defines four services:

- Zookeeper — Kafka requires Zookeeper for coordination. Runs on port 2181.
- Kafka Broker — handles the message stream between producer and consumer. Runs on port 9092.
- MySQL ODB — stores raw transaction records as they come in from the consumer. Mapped to port 13306.
- MySQL DW — a separate MySQL instance for the data warehouse. Keeping it separate from the ODB made schema management cleaner. Mapped to port 23306.

docker-compose.yml:

```
version: '3.8'
services:
```

```

zookeeper:
  image: confluentinc/cp-zookeeper:6.0.1
  platform: linux/amd64
  ports:
    - "2181:2181"
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
broker:
  image: confluentinc/cp-kafka:6.0.1
  platform: linux/amd64
  ports:
    - "9092:9092"
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
mysql-odb:
  image: mysql:8.0
  platform: linux/amd64
  environment:
    MYSQL_ROOT_PASSWORD: secret
    MYSQL_DATABASE: odb
  ports:
    - "13306:3306"
mysql-dw:
  image: mysql:8.0
  platform: linux/amd64
  environment:
    MYSQL_ROOT_PASSWORD: secret
    MYSQL_DATABASE: dw
  ports:
    - "23306:3306"

```

4. Star Schema Design

We used a standard star schema for the warehouse — one central fact table with dimension tables surrounding it. The fact table holds the measurable data (sales, quantity, profit) and the dimension tables hold the descriptive context (who, what, where, when).

Fact Table: sales_fact

Column	Type	Description
sales_key	INT AUTO_INCREMENT PK	Unique row identifier

date_key	INT FK	Links to date_dim
customer_key	INT FK	Links to customer_dim
product_key	INT FK	Links to product_dim
location_key	INT FK	Links to location_dim
quantity	INT	Units ordered
sales_amount	DECIMAL(10,2)	Revenue from transaction
profit_amount	DECIMAL(10,2)	Profit from transaction

Dimension Tables

customer_dim — one row per unique customer name. product_dim — one row per unique product name and category combination. location_dim — one row per unique city, state, and region combination. date_dim — stores date breakdowns by year and month, keyed by a computed integer (YYYYMMDD format).

DDL Statements

```
CREATE TABLE date_dim (
    date_key INT PRIMARY KEY,
    full_date DATE,
    year INT,
    month INT
);
```

```
CREATE TABLE customer_dim (
    customer_key INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100)
);
```

```
CREATE TABLE product_dim (
    product_key INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50)
);
```

```
CREATE TABLE location_dim (
    location_key INT AUTO_INCREMENT PRIMARY KEY,
    city VARCHAR(50),
    state VARCHAR(50),
    region VARCHAR(50)
);
```

```

CREATE TABLE sales_fact (
    sales_key INT AUTO_INCREMENT PRIMARY KEY,
    date_key INT,
    customer_key INT,
    product_key INT,
    location_key INT,
    quantity INT,
    sales_amount DECIMAL(10,2),
    profit_amount DECIMAL(10,2)
);

```

SQL to Populate the Star Schema

```

INSERT INTO customer_dim (customer_name)
SELECT DISTINCT customer_name FROM odb.sales_odb;

```

```

INSERT INTO product_dim (product_name, category)
SELECT DISTINCT product_name, category FROM odb.sales_odb;

```

```

INSERT INTO location_dim (city, state, region)
SELECT DISTINCT city, state, region FROM odb.sales_odb;

```

```

INSERT INTO date_dim (date_key, full_date, year, month)
SELECT DISTINCT
    YEAR(order_date)*10000 + MONTH(order_date)*100 + DAY(order_date),
    order_date, YEAR(order_date), MONTH(order_date)
FROM odb.sales_odb;

```

```

INSERT INTO sales_fact (date_key, customer_key, product_key,
    location_key, quantity, sales_amount, profit_amount)
SELECT
    YEAR(o.order_date)*10000 + MONTH(o.order_date)*100 +
DAY(o.order_date),
    c.customer_key, p.product_key, l.location_key,
    o.quantity, o.sales, o.profit
FROM odb.sales_odb o
JOIN customer_dim c ON o.customer_name = c.customer_name
JOIN product_dim p ON o.product_name = p.product_name
    AND o.category = p.category
JOIN location_dim l ON o.city = l.city AND o.state = l.state
    AND o.region = l.region;

```

5. Data Streams (Kafka)

We used Kafka to simulate real-time ingestion. The pipeline has two parts — a producer that reads from CSV and publishes to a topic, and a consumer that reads from the topic and inserts into MySQL.

Producer (producer.py)

The producer reads the CSV using pandas and sends each row as a JSON message to the sales_topic Kafka topic. We added a short delay between messages to simulate streaming, and limited the run to 100 records for testing.

```
producer = KafkaProducer(  
    bootstrap_servers='localhost:9092',  
    value_serializer=lambda v: json.dumps(v).encode('utf-8')  
)  
for _, row in df.head(100).iterrows():  
    producer.send('sales_topic', row.to_dict())  
    print('Sent:', row['Order ID'])  
    time.sleep(0.1)
```

Consumer (consumer.py)

The consumer subscribes to sales_topic and inserts each message into the sales_odb table. The date conversion also happens here before the INSERT statement runs.

```
for msg in consumer:  
    data = msg.value  
    formatted_date = datetime.strptime(  
        data.get('Order Date'), '%m/%d/%Y').strftime('%Y-%m-%d')  
    cursor.execute("INSERT INTO sales_odb ...", (...))  
    conn.commit()  
    print('Inserted:', data.get('Order ID'))
```

6. Summary Tables

We created pre-aggregated summary tables to make common queries faster. Instead of joining multiple tables every time you want totals by region or category, these tables store the results ahead of time.

Sales by Region

```
CREATE TABLE summary_sales_by_region AS  
SELECT l.region, SUM(f.sales_amount) AS total_sales
```

```
FROM sales_fact f
JOIN location_dim l ON f.location_key = l.location_key
GROUP BY l.region;
```

Profit by Category

```
CREATE TABLE summary_profit_by_category AS
SELECT p.category, SUM(f.profit_amount) AS total_profit
FROM sales_fact f
JOIN product_dim p ON f.product_key = p.product_key
GROUP BY p.category;
```

Monthly Sales

```
CREATE TABLE summary_monthly_sales AS
SELECT d.year, d.month, SUM(f.sales_amount) AS total_sales
FROM sales_fact f
JOIN date_dim d ON f.date_key = d.date_key
GROUP BY d.year, d.month;
```

7. OLAP Queries

We wrote four OLAP queries to demonstrate the analytical capabilities of the warehouse.

Query 1: Total Sales by Region

Breaks down total revenue by region — useful for seeing which parts of the country are driving the most sales.

```
SELECT l.region, SUM(f.sales_amount) AS total_sales
FROM sales_fact f
JOIN location_dim l ON f.location_key = l.location_key
GROUP BY l.region;
```

Query 2: Profit by Category

Shows how profitable each product category is. Helps answer whether Technology is more profitable than Furniture on a per-sale basis.

```
SELECT p.category, SUM(f.profit_amount) AS total_profit
FROM sales_fact f
JOIN product_dim p ON f.product_key = p.product_key
GROUP BY p.category;
```

Query 3: Monthly Sales Trend

Tracks total sales by month and year — useful for spotting seasonal patterns or growth trends over time.

```
SELECT d.year, d.month, SUM(f.sales_amount) AS total_sales
FROM sales_fact f
JOIN date_dim d ON f.date_key = d.date_key
GROUP BY d.year, d.month
ORDER BY d.year, d.month;
```

Query 4: Top 10 Customers by Sales

Ranks the highest-spending customers — useful for identifying valuable customers or targeting loyalty programs.

```
SELECT c.customer_name, SUM(f.sales_amount) AS total_sales
FROM sales_fact f
JOIN customer_dim c ON f.customer_key = c.customer_key
GROUP BY c.customer_name
ORDER BY total_sales DESC
LIMIT 10;
```

8. Screenshots and Results

The screenshots below walk through the pipeline from data ingestion to final query results.

Screenshot 1: Kafka producer streaming records from the Superstore CSV into the Kafka topic. Each 'Sent' message confirms a record was successfully published.

[Insert Screenshot 1 — Producer output]

Screenshot 2: Kafka consumer receiving messages and inserting data into the MySQL ODB in real time.

[Insert Screenshot 2 — Consumer output]

Screenshot 3: Sample records from the operational database (sales_odb) confirming the pipeline loaded data correctly.

[Insert Screenshot 3 — ODB records]

Screenshots 4 & 5: Row counts across all data warehouse tables confirming successful loading of dimension and fact tables.

[Insert Screenshots 4 & 5 — DW row counts]

Screenshot 6: OLAP query output showing total sales by region.

[Insert Screenshot 6 — Sales by region query result]

Conclusion

This project gave us a solid end-to-end look at how a data engineering pipeline actually works in practice. Going from a raw CSV to a structured data warehouse with analytical queries running on top of it has a lot more moving parts than it sounds like, and getting Kafka, MySQL, and the Python scripts all working together took some real debugging.

A few things stood out during the project:

- The date formatting issue was a small thing that could have caused a lot of pain if we hadn't caught it early. It's a good reminder that data cleaning matters even when the dataset looks clean at first glance.
- Having separate MySQL containers for the ODB and DW made everything easier to manage and also better reflects how these systems are actually set up in production environments.
- The star schema made the OLAP queries really straightforward to write. Once the fact and dimension tables are designed correctly, the queries almost write themselves.

If we were to extend the project, we'd probably stream the full 9,994 records instead of just 100, add more OLAP queries (like discount impact analysis or sub-category breakdowns), and look at setting up a simple visualization layer on top of the warehouse.

References

Superstore Dataset. Kaggle. <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

Apache Kafka Documentation. <https://kafka.apache.org/documentation/>

MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

Confluent Platform Docker Images. <https://hub.docker.com/u/confluentinc>

Lab 7 Course Materials — INFSCI 1540, University of Pittsburgh